

CO4 AT1

INDUSTRY PROBLEM-SOLVING TASK

NAME:CH.HARSHITHA

REGNO:192473031

CODE:CSA0905RA

COURSE:JAVA PROGRAMMING

Employee Performance & Payroll Dashboard

Problem Statement: A software company wants to develop a simple Employee Performance and Payroll Management System using Java AWT. The application should allow the HR team to enter employee details, process salary information, and generate a structured payroll report.

CODE:

```
import java.awt.*;
```

```
import java.awt.event.*;
```

```
public class EmployeePayroll extends Frame implements ActionListener {
```

```
    TextField empId, empName, salary, workingDays;
```

```
    Choice department, designation, performance;
```

```
    Checkbox fullTime, partTime;
```

```
    CheckboxGroup employeeTypeGroup;
```

```
    Button generate, reset;
```

```
    TextArea report;
```

```
EmployeePayroll() {

    setTitle("Employee Performance & Payroll Dashboard");

    setSize(700, 650);

    setLayout(new GridBagLayout());

    GridBagConstraints gbc = new GridBagConstraints();

    gbc.insets = new Insets(6, 6, 6, 6);

    gbc.fill = GridBagConstraints.HORIZONTAL;

    gbc.gridx = 0;

    gbc.gridy = 0;

    add(new Label("Employee ID:"), gbc);

    gbc.gridx = 1;

    empId = new TextField(25);

    add(empId, gbc);

    gbc.gridx = 0;

    gbc.gridy = 1;

    add(new Label("Employee Name:"), gbc);

    gbc.gridx = 1;

    empName = new TextField(25);

    add(empName, gbc);

    gbc.gridx = 0;
```

```
gbc.gridy = 2;  
add(new Label("Department:"), gbc);
```

```
gbc.gridx = 1;  
department = new Choice();  
department.add("CSE");  
department.add("ECE");  
department.add("IT");  
department.add("HR");  
department.add("Finance");  
add(department, gbc);
```

```
gbc.gridx = 0;  
gbc.gridy = 3;  
add(new Label("Designation:"), gbc);
```

```
gbc.gridx = 1;  
designation = new Choice();  
designation.add("Software Engineer");  
designation.add("Senior Software Engineer");  
designation.add("Team Lead");  
designation.add("Project Manager");  
add(designation, gbc);
```

```
gbc.gridx = 0;  
gbc.gridy = 4;  
add(new Label("Employee Type:"), gbc);
```

```
gbc.gridx = 1;
```

```
employeeTypeGroup = new CheckboxGroup();
```

```
fullTime = new Checkbox("Full Time", employeeTypeGroup, true);
```

```
partTime = new Checkbox("Part Time", employeeTypeGroup, false);
```

```
Panel typePanel = new Panel();
```

```
typePanel.add(fullTime);
```

```
typePanel.add(partTime);
```

```
add(typePanel, gbc);
```

```
gbc.gridx = 0;
```

```
gbc.gridy = 5;
```

```
add(new Label("Basic Salary:"), gbc);
```

```
gbc.gridx = 1;
```

```
salary = new TextField(25);
```

```
add(salary, gbc);
```

```
gbc.gridx = 0;
```

```
gbc.gridy = 6;
```

```
add(new Label("Working Days:"), gbc);
```

```
gbc.gridx = 1;
```

```
workingDays = new TextField(25);
```

```
add(workingDays, gbc);
```

```
gbc.gridx = 0;  
gbc.gridy = 7;  
add(new Label("Performance Rating:"), gbc);
```

```
gbc.gridx = 1;  
performance = new Choice();  
performance.add("Excellent");  
performance.add("Good");  
performance.add("Average");  
performance.add("Poor");  
add(performance, gbc);
```

```
gbc.gridx = 0;  
gbc.gridy = 8;  
generate = new Button("Generate Report");  
add(generate, gbc);
```

```
gbc.gridx = 1;  
reset = new Button("Reset");  
add(reset, gbc);
```

```
gbc.gridx = 0;  
gbc.gridy = 9;  
gbc.gridwidth = 2;  
gbc.fill = GridBagConstraints.BOTH;  
gbc.weightx = 1;  
gbc.weighty = 1;
```

```

report = new TextArea(12, 55);

report.setEditable(false);

add(report, gbc);


generate.addActionListener(this);

reset.addActionListener(this);


addWindowListener(new WindowAdapter() {

    public void windowClosing(WindowEvent e) {

        System.exit(0);

    }

});


setVisible(true);

}


public void actionPerformed(ActionEvent e) {

    if (e.getSource() == generate) {

        if (empId.getText().trim().isEmpty() ||

            empName.getText().trim().isEmpty() ||

            salary.getText().trim().isEmpty() ||

            workingDays.getText().trim().isEmpty()) {

            report.setText("Please fill all required fields.");

            return;

```

```
}
```

```
try {
```

```
    double basicSalary = Double.parseDouble(salary.getText().trim());
```

```
    int days = Integer.parseInt(workingDays.getText().trim());
```

```
    if (basicSalary <= 0) {
```

```
        report.setText("Basic salary must be greater than 0.");
```

```
        return;
```

```
    }
```

```
    if (days <= 0 || days > 31) {
```

```
        report.setText("Working days must be between 1 and 31.");
```

```
        return;
```

```
    }
```

```
    double grossSalary = basicSalary;
```

```
    double bonusPercentage;
```

```
    if (performance.getSelectedItem().equals("Excellent")) {
```

```
        bonusPercentage = 20;
```

```
    } else if (performance.getSelectedItem().equals("Good")) {
```

```
        bonusPercentage = 10;
```

```
    } else if (performance.getSelectedItem().equals("Average")) {
```

```
        bonusPercentage = 5;
```

```
    } else {
```

```
        bonusPercentage = 0;
```

```
}
```

```
double performanceBonus =  
    basicSalary * bonusPercentage / 100;
```

```
double netSalary =  
    grossSalary + performanceBonus;
```

```
String employeeTypeValue =  
    employeeTypeGroup.getSelectedCheckbox().getLabel();
```

```
report.setText(  
    "=====\n" +  
    "    EMPLOYEE PAYROLL REPORT\n" +  
    "=====\n\n" +  
    "Employee ID    : " + empId.getText() + "\n" +  
    "Employee Name   : " + empName.getText() + "\n" +  
    "Department     : " + department.getSelectedItem() + "\n" +  
    "Designation    : " + designation.getSelectedItem() + "\n" +  
    "Employee Type   : " + employeeTypeValue + "\n" +  
    "Basic Salary   : Rs. " + basicSalary + "\n" +  
    "Working Days   : " + days + "\n" +  
    "Performance    : " + performance.getSelectedItem() + "\n" +  
    "-----\n" +  
    "Gross Salary   : Rs. " + grossSalary + "\n" +  
    "Bonus Percentage : " + bonusPercentage + "%\n" +  
    "Performance Bonus : Rs. " + performanceBonus + "\n" +  
    "Net Salary     : Rs. " + netSalary + "\n" +
```



```

        "=====
    );

    } catch (NumberFormatException ex) {

        report.setText(

            "Invalid input!\n" +

            "Basic Salary and Working Days must be numeric."

        );
    }
}

if (e.getSource() == reset) {

    empId.setText("");
    empName.setText("");
    salary.setText("");
    workingDays.setText("");

    department.select(0);
    designation.select(0);
    performance.select(0);

    employeeTypeGroup.setSelectedCheckbox(fullTime);

    report.setText("");
}
}

```

```

public static void main(String[] args) {

    new EmployeePayroll();

}

```

The screenshot shows a Java Swing window titled "Employee Performance & Payroll Dashboard". The window contains a form with the following fields and controls:

- Employee ID: Text field with value "435"
- Employee Name: Text field with value "HARSHI"
- Department: Dropdown menu with value "CSE"
- Designation: Dropdown menu with value "Software Engineer"
- Employee Type: Radio buttons for "Full Time" (selected) and "Part Time"
- Basic Salary: Text field with value "20000"
- Working Days: Text field with value "30"
- Performance Rating: Dropdown menu with value "Excellent"
- Buttons: "Generate Report" and "Reset"

Below the form, there is a scrollable text area displaying the "EMPLOYEE PAYROLL REPORT" for the entered data:

```

=====
EMPLOYEE PAYROLL REPORT
=====
Employee ID   : 435
Employee Name : HARSHI
Department    : CSE
Designation   : Software Engineer
Employee Type  : Full Time
Basic Salary  : Rs. 20000.0
Working Days  : 30
Performance   : Excellent
-----
Gross Salary  : Rs. 20000.0
Bonus Percentage : 20.0%
Performance Bonus : Rs. 4000.0
Net Salary    : Rs. 24000.0
=====

```

```

}

```

OUTPUT:

Online Food Ordering and Billing System

Problem Statement: A restaurant wants to develop a simple Food Ordering and Billing Application for its counter staff. The application should allow customers to select food items, specify quantities, and generate a final bill.

CODE:

```
import java.awt.*;
```

```
import java.awt.event.*;
```

```
import java.util.ArrayList;
```

```
public class FoodOrdering extends Frame implements ActionListener {
```

```
    CardLayout cardLayout;
```

```
    Panel cards;
```

```
    TextField customerName, tableNumber, quantity;
```

```
    Choice category, foodItem;
```

```
    List orderList;
```

```
    TextArea billArea;
```

```
    Button addItem, generateBill, clear, nextCustomer, previousFood, nextFood, previousBill, clearBill;
```

```
    ArrayList<String> items = new ArrayList<>();
```

```
    ArrayList<Integer> quantities = new ArrayList<>();
```

```
    ArrayList<Double> prices = new ArrayList<>();
```

```
    FoodOrdering() {
```

```
        setTitle("Online Food Ordering and Billing System");
```

```
        setSize(700, 600);
```

```
setLayout(new BorderLayout());
```

```
cardLayout = new CardLayout();
```

```
cards = new Panel(cardLayout);
```

```
Panel card1 = new Panel(new GridLayout(5, 2, 10, 10));
```

```
card1.add(new Label("Customer Name:"));
```

```
customerName = new TextField();
```

```
card1.add(customerName);
```

```
card1.add(new Label("Table Number:"));
```

```
tableNumber = new TextField();
```

```
card1.add(tableNumber);
```

```
card1.add(new Label("Food Category:"));
```

```
category = new Choice();
```

```
category.add("Starters");
```

```
category.add("Main Course");
```

```
category.add("Beverages");
```

```
category.add("Desserts");
```

```
card1.add(category);
```

```
card1.add(new Label(""));
```

```
card1.add(new Label(""));
```

```
nextCustomer = new Button("Next");
```

```
clear = new Button("Clear");
```

```
card1.add(nextCustomer);
```

```
card1.add(clear);
```

```
cards.add(card1, "Customer");
```

```
Panel card2 = new Panel(new GridLayout(7, 2, 10, 10));
```

```
card2.add(new Label("Food Category:"));
```

```
Choice categoryDisplay = new Choice();
```

```
categoryDisplay.add("Starters");
```

```
categoryDisplay.add("Main Course");
```

```
categoryDisplay.add("Beverages");
```

```
categoryDisplay.add("Desserts");
```

```
card2.add(categoryDisplay);
```

```
card2.add(new Label("Food Item:"));
```

```
foodItem = new Choice();
```

```
foodItem.add("Veg Burger - 120");
```

```
foodItem.add("Pizza - 250");
```

```
foodItem.add("French Fries - 100");
```

```
foodItem.add("Chicken Biryani - 220");
```

```
foodItem.add("Fried Rice - 180");
```

```
foodItem.add("Coke - 60");
```

```
foodItem.add("Fresh Juice - 80");
```

```
foodItem.add("Ice Cream - 90");
```

```
foodItem.add("Gulab Jamun - 70");
```

```
card2.add(foodItem);
```

```
card2.add(new Label("Quantity:"));
```

```
quantity = new TextField();
```

```
card2.add(quantity);
```

```
addItem = new Button("Add Item");
```

```
card2.add(addItem);
```

```
generateBill = new Button("Generate Bill");
```

```
card2.add(generateBill);
```

```
previousFood = new Button("Previous");
```

```
card2.add(previousFood);
```

```
nextFood = new Button("Next");
```

```
card2.add(nextFood);
```

```
orderList = new List(10);
```

```
card2.add(orderList);
```

```
cards.add(card2, "Food");
```

```
Panel card3 = new Panel(new BorderLayout());
```

```
billArea = new TextArea();
```

```
billArea.setEditable(false);
```

```
card3.add(new Label("BILL / ORDER SUMMARY", Label.CENTER),
```

```
BorderLayout.NORTH);
```

```
card3.add(billArea, BorderLayout.CENTER);
```

```
Panel bottom = new Panel();
```

```
previousBill = new Button("Previous");
```

```
clearBill = new Button("Clear");
```

```
bottom.add(previousBill);
```

```
bottom.add(clearBill);
```

```
card3.add(bottom, BorderLayout.SOUTH);
```

```
cards.add(card3, "Bill");
```

```
add(cards, BorderLayout.CENTER);
```

```
nextCustomer.addActionListener(this);
```

```
clear.addActionListener(this);
```

```
addItem.addActionListener(this);
```

```
generateBill.addActionListener(this);
```

```
previousFood.addActionListener(this);
```

```
nextFood.addActionListener(this);
```

```
previousBill.addActionListener(this);
```

```
clearBill.addActionListener(this);
```

```
category.addItemListener(new ItemListener() {
```

```

        public void itemStateChanged(ItemEvent e) {

            categoryDisplay.select(category.getSelectedItem());

        }

    });

setVisible(true);

addWindowListener(new WindowAdapter() {

    public void windowClosing(WindowEvent e) {

        System.exit(0);

    }

});

}

public void actionPerformed(ActionEvent e) {

    if (e.getSource() == nextCustomer) {

        if (customerName.getText().trim().isEmpty() ||

            tableNumber.getText().trim().isEmpty()) {

            billArea.setText(

                "Please enter Customer Name and Table Number."

            );

            cardLayout.show(cards, "Customer");

            return;

        }
    }
}

```



```
cardLayout.show(cards, "Food");  
}
```

```
if (e.getSource() == addItem) {
```

```
    if (quantity.getText().trim().isEmpty()) {
```

```
        orderList.add("Please enter quantity.");  
        return;  
    }
```

```
    try {
```

```
        int qty = Integer.parseInt(quantity.getText().trim());
```

```
        if (qty <= 0) {
```

```
            orderList.add(  
                "Quantity must be greater than 0."  
            );
```

```
            return;  
        }
```

```
        String selectedItem =
```

```
            foodItem.getSelectedItem();
```

```

        double price =
            getPrice(selectedItem);

        items.add(selectedItem);

        quantities.add(qty);

        prices.add(price);

        double total = price * qty;

        orderList.add(
            selectedItem +
            " x " +
            qty +
            " = Rs." +
            total
        );

        quantity.setText("");

    } catch (NumberFormatException ex) {

        orderList.add(
            "Quantity must be a number."
        );
    }
}

if (e.getSource() == generateBill) {

```

```
if (items.size() == 0) {  
  
    billArea.setText(  
        "Please add at least one food item."  
    );  
  
    cardLayout.show(cards, "Food");  
    return;  
}
```

```
generateBillReport();
```

```
cardLayout.show(cards, "Bill");  
}
```

```
if (e.getSource() == previousFood) {  
    cardLayout.show(cards, "Customer");  
}
```

```
if (e.getSource() == nextFood) {
```

```
    if (items.size() == 0) {  
  
        orderList.add(  
            "Please add at least one item."  
        );  
    }
```

```
        return;
    }

    generateBillReport();

    cardLayout.show(cards, "Bill");
}

if (e.getSource() == previousBill) {
    cardLayout.show(cards, "Food");
}

if (e.getSource() == clear ||
    e.getSource() == clearBill) {

    clearAll();
}
}

double getPrice(String item) {

    if (item.equals("Veg Burger - 120"))
        return 120;

    if (item.equals("Pizza - 250"))
        return 250;

    if (item.equals("French Fries - 100"))
```

```
        return 100;

        if (item.equals("Chicken Biryani - 220"))

            return 220;

        if (item.equals("Fried Rice - 180"))

            return 180;

        if (item.equals("Coke - 60"))

            return 60;

        if (item.equals("Fresh Juice - 80"))

            return 80;

        if (item.equals("Ice Cream - 90"))

            return 90;

        if (item.equals("Gulab Jamun - 70"))

            return 70;

        return 0;
    }
}
```

```
void generateBillReport() {
```

```
    double subtotal = 0;
```

```
    StringBuilder bill = new StringBuilder();
```

```
bill.append("=====\\n");
```

```
bill.append("    FOOD BILL\\n");
```

```
bill.append("=====\\n");
```

```
bill.append("Customer Name : ");
```

```
bill.append(customerName.getText());
```

```
bill.append("\\n");
```

```
bill.append("Table Number : ");
```

```
bill.append(tableNumber.getText());
```

```
bill.append("\\n");
```

```
bill.append("-----\\n");
```

```
for (int i = 0; i < items.size(); i++) {
```

```
    double total =
```

```
        prices.get(i) *
```

```
        quantities.get(i);
```

```
    subtotal = subtotal + total;
```

```
    bill.append(items.get(i));
```

```
    bill.append(" x ");
```

```
    bill.append(quantities.get(i));
```

```
    bill.append(" = Rs.");
```

```
    bill.append(total);
```

```

        bill.append("\n");
    }

    double gst = subtotal * 0.05;

    double finalAmount = subtotal + gst;

    bill.append("-----\n");

    bill.append("Subtotal    : Rs.");
    bill.append(subtotal);
    bill.append("\n");

    bill.append("GST (5%)    : Rs.");
    bill.append(gst);
    bill.append("\n");

    bill.append("Final Bill   : Rs.");
    bill.append(finalAmount);
    bill.append("\n");

    bill.append("===== \n");
    bill.append("    THANK YOU! \n");
    bill.append("=====");

    billArea.setText(bill.toString());
}

void clearAll() {

```

```
customerName.setText("");
```

```
tableNumber.setText("");
```

```
quantity.setText("");
```

```
orderList.removeAll();
```

```
items.clear();
```

```
quantities.clear();
```

```
prices.clear();
```

```
billArea.setText("");
```

```
cardLayout.show(cards, "Customer");
```

```
}
```

```
public static void main(String[] args) {
```

```
    new FoodOrdering();
```

```
}
```

```
}
```

OUTPUT:

Online Food Ordering and Billing System

Customer Name:

HARSHITHA

Table Number:

4

Food Category:

Starters

Next

Clear

Online Food Ordering and Billing System

Food Category:

Starters

Food Item:

Veg Burger - 120

Quantity:

Add Item

Generate Bill

Previous

Next

Veg Burger - 120 x 1 = Rs.120.0
Veg Burger - 120 x 2 = Rs.240.0

BILL / ORDER SUMMARY

=====

FOOD BILL

=====

Customer Name : HARSHITHA

Table Number : 4

Veg Burger - 120 x 1 = Rs.120.0

Veg Burger - 120 x 2 = Rs.240.0

Subtotal : Rs.360.0

GST (5%) : Rs.18.0

Final Bill : Rs.378.0

=====

THANK YOU!

=====

Previous

Clear